

10__n-gram_models_filled

May 10, 2026

1 CSE 25 - Introduction to Artificial Intelligence

1.1 Week 7 Tuesday: n-gram language models

Learning Objectives:

- Explain the Markov assumption in n-gram language models
- Compute unigram and bigram probabilities from frequency counts
- Explain sparsity and apply add α smoothing
- Compute and interpret log-probability and perplexity

Instructions

Use your copy of this notebook on Datahub and complete it during class. Work through the cells below **in order**. You may discuss with your neighbors, but make sure you understand each step yourself.

SUBMISSION: When finished, download the notebook to have a local copy for your records. Choose two cells where you wrote code or answers and take screenshots of them to upload to Gradescope under **In-Class - Week 7 Tuesday** to receive credit.

Language Model A *language model* over a vocabulary V assigns probabilities to strings drawn from V^* ,

Let's say we want to compute the probability of the next word given some history:

$$P(w \mid h)$$

Suppose the history is:

On summer evenings the sky looks very

and we want the probability that the next word is *orange*:

$$P(\text{orange} \mid \text{On summer evenings the sky looks very})$$

A simple idea is to estimate this using counts from a large corpus.

We count:

- How often we see the full sequence
On summer evenings the sky looks very orange

- How often we see the history
On summer evenings the sky looks very

This gives the relative-frequency estimate:

$$P(w \mid h) = \frac{C(hw)}{C(h)}$$

In words:

Out of all the times we saw the history h , how often was it followed by the word w ?

Now suppose we want the probability of an entire sequence of words rather than just one next word.

Using the relationship between conditional and joint probability, we can write:

$$\begin{aligned} P(w_1, w_2, \dots, w_n) &= P(w_1) \cdot P(w_2 \mid w_1) \cdot P(w_3 \mid w_1, w_2) \cdots P(w_n \mid w_1, \dots, w_{n-1}) \\ &= \prod_{k=1}^n P(w_k \mid w_1, \dots, w_{k-1}) \end{aligned}$$

Language Modeling with N-grams Computing $P(w_n \mid w_1, w_2, \dots, w_{n-1})$ for long histories is unrealistic in practice.

To make the problem tractable, we deliberately simplify the model.

Instead of conditioning on the *entire* history, we approximate it using only *the last few words*.

An **n-gram language model** makes the simplifying assumption that each word depends only on the previous $n - 1$ words:

$$P(w_k \mid w_1, \dots, w_{k-1}) \approx P(w_k \mid w_{k-n+1}, \dots, w_{k-1})$$

Using this assumption, the probability of a sequence becomes:

$$P(w_1, w_2, \dots, w_n) \approx \prod_{k=1}^n P(w_k \mid w_{k-n+1}, \dots, w_{k-1})$$

- This simplification is called a **Markov assumption**. It means that the future depends only on a limited recent past, not the entire history.
- This also introduces **position invariance**. In an n-gram model, the probability assigned to a word given a specific local context is the same no matter where that context appears in the sentence.
- The conditional probabilities $P(w_k \mid w_{k-n+1}, \dots, w_{k-1})$ are the parameters of the model that we estimate from data.

Models:

- **Unigram model** ($n = 1$): $P(w_k \mid w_1, \dots, w_{k-1}) = P(w_k)$
- **Bigram model** ($n = 2$): $P(w_k \mid w_1, \dots, w_{k-1}) = P(w_k \mid w_{k-1})$

- **Trigram model** ($n = 3$): $P(w_k | w_1, \dots, w_{k-1}) = P(w_k | w_{k-2}, w_{k-1})$

Maximum Likelihood Estimation Once we decide how much history to use (unigram, bigram, trigram, etc.), we need a way to compute the probabilities from data.

We use **Maximum Likelihood Estimation (MLE)**: Choose the probabilities that make the observed data most likely, i.e. maximize the likelihood of the data.

What Do We Mean by “Likelihood of the Data”?

Suppose our corpus (our dataset of text) is a sequence of N tokens: $w_1, w_2, \dots, w_N$

An n-gram model assigns a probability to the **entire corpus**:

$$P(\text{corpus}) = P(w_1, w_2, \dots, w_N) = \prod_{k=1}^N P(w_k | w_{k-n+1}, \dots, w_{k-1})$$

This quantity is called the **likelihood** of the data under the model.

- The corpus is fixed.
- The probabilities are the parameters we will compute.
- MLE selects the probabilities that **maximize this product**.

For n-gram models, MLE leads to **relative frequency estimates**.

For a **unigram model**:

$$P(w_k) = \frac{C(w_k)}{\text{total number of words in the corpus}}$$

Count how often the word appears,
divide by the total number of tokens.

For a **bigram model**:

$$P(w_k | w_{k-1}) = \frac{C(w_{k-1}, w_k)}{C(w_{k-1})}$$

Count how often the two-word sequence appears,
divide by how often the first word appears.

For a **trigram model**:

$$P(w_k | w_{k-2}, w_{k-1}) = \frac{C(w_{k-2}, w_{k-1}, w_k)}{C(w_{k-2}, w_{k-1})}$$

Count how often the three-word sequence appears,
divide by how often the two-word history appears.

In general, for an n-gram model:

$$P(w_k \mid w_{k-n+1}, \dots, w_{k-1}) = \frac{C(w_{k-n+1}, \dots, w_k)}{C(w_{k-n+1}, \dots, w_{k-1})}$$

So computing n-gram probabilities always follows the same pattern:

1. Count the full sequence (history + next word).
2. Divide by the count of the history.

1.1.1 Example language model

We will work through generating a language model based on a tiny example, so each computation will be transparent and concrete.

The corpus consists of sentences.

Step 1: tokenization. Step 2: vocabulary extraction. Step 3: Estimate probabilities (for chosen n-gram). Step 4: Add smoothing. Step 5: Evaluate model.

Tokenization is the process of splitting text into smaller units called *tokens*. In practice, tokens can be words, subwords, characters, or punctuation, depending on the tokenizer and model.

For this toy exercise, we will treat words as tokens and include boundary markers:

- <s> start of sentence
- </s> end of sentence

Vocabulary is the set of all **unique tokens** that appear in the corpus. For this toy corpus, the vocabulary includes words and boundary tokens such as <s> and </s>. Its size is denoted by $|V|$.

```
[13]: # Tokenized sentences
tokenized_toy_sentences = [
    ["<s>", "to", "be", "or", "not", "to", "be", "</s>"],
    ["<s>", "to", "be", "a", "king", "</s>"],
    ["<s>", "to", "eat", "pizza", "</s>"],
]

all_tokens = []

# Create a list of all tokens in the corpus
for sentence in tokenized_toy_sentences:
    for token in sentence:
        all_tokens.append(token)

N = len(all_tokens)
print("Total Tokens, N:", N)
print("Tokens:", all_tokens)
```

Total Tokens, N: 19

Tokens: ['<s>', 'to', 'be', 'or', 'not', 'to', 'be', '</s>', '<s>', 'to', 'be', 'a', 'king', '</s>', '<s>', 'to', 'eat', 'pizza', '</s>']

```
[14]: # Vocabulary - the set of unique tokens in the corpus

# Create a set of unique words to form the vocabulary:
#REMOVE FROM STUDENT VERSION
vocab = set(all_tokens)
#vocab = None # YOUR CODE HERE

# Sort the set of tokens alphabetically for cleaner display
vocab_order = sorted(vocab)
# Get the size of the vocabulary
vocab_size = len(vocab)

print("Vocabulary, V:", vocab)
print("Vocab Size, |V|:", vocab_size)
```

Vocabulary, V: {'be', 'to', 'a', 'not', 'pizza', '<s>', '</s>', 'eat', 'or', 'king'}

Vocab Size, |V|: 10

Q. If the vocabulary size is $|V|$, how many parameters do we need to compute for:

- Unigram Model - YOUR ANSWER HERE

Hint need $P(w_k)$ for each w_k

- Bigram Model - YOUR ANSWER HERE

Hint need $P(w_k | w_{k-1})$ for each w_k and w_{k-1}

- Trigram Model - YOUR ANSWER HERE

Hint need $P(w_k | w_{k-1}, w_{k-2})$ for each w_k and w_{k-1} and w_{k-2}

Hint for all three: A “parameter” is one entry in the probability table you are about to build. For the unigram model, that table has one row per vocabulary word. For the bigram model, the table has one row and one column per vocabulary word (a matrix). For the trigram model, think one level higher.

```
[15]: # Let's estimate unigram probabilities for all_tokens
# For each token in the vocabulary, we count its occurrences in the corpus
# and divide by the total number of tokens in the corpus (N)

# Note: it's more efficient to iterate through corpus once and update the counts
# for all vocabulary tokens

# Initialize a dictionary to count the occurrences of each word
word_counts = {}
for w in all_tokens:
    # If it's the first time we see this word, initialize its count to 1
    if w not in word_counts:
```

```

#REMOVE FROM STUDENT VERSION
word_counts[w] = 1 # YOUR CODE HERE

#word_counts[w] = None # YOUR CODE HERE
# else, increment the existing count by 1
else:
    #REMOVE FROM STUDENT VERSION
    word_counts[w] += 1 # YOUR CODE HERE

    #word_counts[w] = None # YOUR CODE HERE

# Now we can compute the unigram probabilities
# by dividing the count of each word by
# the total number of tokens (N)

# Initialize a dictionary to store unigram probabilities
unigram_probs = {}
total_prob = 0.0

# Calculate unigram probabilities for each token in the vocabulary
for token in vocab:
    # Calculate the unigram probability for this token

    #REMOVE FROM STUDENT VERSION
    unigram_probs[token] = word_counts[token] / len(all_tokens) # YOUR CODE HERE

    #unigram_probs[token] = None # YOUR CODE HERE

# Add up total probabilities, for error-checking
total_prob += unigram_probs[token]

print("Unigram probabilities:" )
for w in sorted(unigram_probs, key=unigram_probs.get, reverse=True):
    print(f" {w}: {unigram_probs[w]:.3f}")

assert abs(total_prob - 1.0) < 1e-9, f"Probabilities do not sum to 1: got {total_prob}"
print(f"Total probability: {total_prob:.6f}")

```

Unigram probabilities:

```

to: 0.211
be: 0.158
<s>: 0.158
</s>: 0.158
a: 0.053
not: 0.053
pizza: 0.053
eat: 0.053

```

```
or: 0.053
king: 0.053
Total probability: 1.000000
```

```
[16]: # Now let's estimate bigram probabilities by counting bigrams and dividing by
      ↪ the count of the previous word

bigram_counts = {}

# Initialize bigram counts for all possible bigrams in the vocabulary
for prev_word in vocab:
    for curr_word in vocab:
        # We can use a tuple as the key for bigram counts
        # since tuples are immutable and can be used as dictionary keys
        bigram_counts[(prev_word, curr_word)] = 0 # Initialize count to 0 for
        ↪ all possible bigrams

# Count bigrams in the tokenized sentences (our text corpus)
for sent in tokenized_toys_sentences:
    for i in range(len(sent) - 1):
        prev_word = sent[i] # w_t-1
        curr_word = sent[i + 1] # w_t

        new_key = (prev_word, curr_word) # (w_t-1, w_t)

        # Update bigram counts
        if (prev_word, curr_word) not in bigram_counts:

            #REMOVE FROM STUDENT VERSION
            bigram_counts[(prev_word, curr_word)] = 1 # YOUR CODE HERE

            #bigram_counts[(prev_word, curr_word)] = None # YOUR CODE HERE
        else:

            #REMOVE FROM STUDENT VERSION
            bigram_counts[(prev_word, curr_word)] += 1 # YOUR CODE HERE

            #bigram_counts[(prev_word, curr_word)] = None # YOUR CODE HERE

# print bigram counts as a table
# with rows as previous words and columns as current words
print("\nBigram Counts:")
print(f"{'':>10}", end="")
for curr_word in vocab_order:
    print(f"{curr_word:>10}", end="")
print()
```

```

for prev_word in vocab_order:
    print(f"{prev_word:>10}", end="")
    for curr_word in vocab_order:
        print(f"{bigram_counts[(prev_word, curr_word)]:>10}", end="")
    print()

```

Bigram Counts:

		</s>	<s>	a	be	eat	king	not
or	pizza	to						
	</s>	0	0	0	0	0	0	0
0	0	0						
	<s>	0	0	0	0	0	0	0
0	0	3						
	a	0	0	0	0	0	1	0
0	0	0						
	be	1	0	1	0	0	0	0
1	0	0						
	eat	0	0	0	0	0	0	0
0	1	0						
	king	1	0	0	0	0	0	0
0	0	0						
	not	0	0	0	0	0	0	0
0	0	1						
	or	0	0	0	0	0	0	1
0	0	0						
	pizza	1	0	0	0	0	0	0
0	0	0						
	to	0	0	0	3	1	0	0
0	0	0						

Q: Pick one nonzero entry in the table (one pair of row and column values) and explain, in terms of the text corpus why it has that value.

YOUR ANSWER HERE

Q: Pick one zero in the table (one pair of row and column values) and explain, in terms of the text corpus why it has that value.

YOUR ANSWER HERE

```

[17]: # Now we can compute the bigram probabilities
      # by dividing the count of each bigram by the count of the previous word
      #  $P(w_t | w_{t-1}) = \text{Count}(w_{t-1}, w_t) / \text{Count}(w_{t-1})$ 

      # Note: word_counts[prev_word] will always be > 0 here because our vocabulary
      # is derived from all_tokens, so every vocab word appears at least once.
      # In production code, we can add a check to guard against division by zero.

```



```

bigram_probs = {}
for (prev_word, curr_word) in bigram_counts:

    #REMOVE FROM STUDENT VERSION
    denom = word_counts[prev_word]
    bigram_probs[(prev_word, curr_word)] = bigram_counts[(prev_word,curr_word)]/
    ↪ denom if denom >0 else 0 # YOUR CODE HERE

    #bigram_probs[(prev_word, curr_word)] = None # YOUR CODE HERE

# Print bigram probabilities as a table
# with rows as previous words
# and columns as current words

print("\nBigram Probabilities:")
print(f"{'':>7}", end="")
for curr_word in vocab_order:
    print(f"{curr_word:>7}", end="")
print()
for prev_word in vocab_order:
    print(f"{prev_word:>7}", end="")
    for curr_word in vocab_order:
        print(f"{bigram_probs[(prev_word, curr_word)]:>7.2f}", end="")
    print()

```

Bigram Probabilities:

	</s>	<s>	a	be	eat	king	not	or	pizza	to
</s>	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
<s>	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00
a	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00	0.00	0.00
be	0.33	0.00	0.33	0.00	0.00	0.00	0.00	0.33	0.00	0.00
eat	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00
king	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
not	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00
or	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	0.00	0.00
pizza	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
to	0.00	0.00	0.00	0.75	0.25	0.00	0.00	0.00	0.00	0.00

Q: Does every row sum to 1? Why or why not?

YOUR ANSWER HERE

Q: Does every column sum to 1? Why or why not?

YOUR ANSWER HERE

Q: What does it mean, in terms of the text corpus, when an entry in this table is 1.00?

YOUR ANSWER HERE

```
[18]: import random

def generate_bigram(bigram_probs, vocab, max_tokens=20, seed=42):
    """
    Generate a sequence by sampling from bigram probabilities.
    """
    if seed is not None:
        random.seed(seed)

    current = "<s>"
    tokens = [current]
    for _ in range(max_tokens):
        # Get all possible next words and their probabilities
        next_words = [w for w in vocab]
        probs = [bigram_probs[(current, w)] for w in next_words]

        # Sample the next word
        current = random.choices(next_words, weights=probs, k=1)[0]
        tokens.append(current)

        if current == "</s>":
            break
    return " ".join(tokens)

print(generate_bigram(bigram_probs, vocab, seed=10))
```

<s> to be </s>

```
[19]: # Using the bigram probabilities to estimate the probability of sequences of
    ↪ tokens
    # For a sequence $w_1, w_2, w_3, \dots, w_n$ (using bigram assumption)
    # we estimate the probability as
    #  $P(w_1) P(w_2 | w_1) P(w_3 | w_2) \dots P(w_n | w_{n-1})$ 

def sequence_prob_bigram(tokenized_sequence):
    # Calculate the probability of a sequence using bigram probabilities
    # Initialize the probability of the sequence to 1 (because we will multiply
    ↪ probabilities)
    prob = 1.0
    for i in range(len(tokenized_sequence)-1):
        # We can use the bigram probabilities
        # we computed above to calculate the probability of the sequence

        # Get the previous word and current word to form the key for bigram
    ↪ probabilities
        prev_word = tokenized_sequence[i]
        curr_word = tokenized_sequence[i + 1]
```

```

        # Get the bigram probability for the current bigram (prev_word,
↪curr_word)
        # Multiply the probabilities of each bigram in the sequence to get the
↪overall sequence probability

#REMOVE FROM STUDENT VERSION
prob = prob * bigram_probs[(prev_word,curr_word)] # YOUR CODE HERE

#prob = None # YOUR CODE HERE

return prob

```

Q: How does initializing the probability of a sequence to 1 connect with the definition that

$$P(w_1, w_2, w_3, \dots, w_n) \approx P(w_1) \cdot P(w_2|w_1) \cdots P(w_n|w_{n-1})$$

YOUR ANSWER HERE

This is a very small corpus so the non-zero bigram probabilities are relatively large (0.25, 0.5, etc.)

With a larger corpus, the bigram probabilities will be much smaller, and the probability of a sentence will be much smaller as well

Let's calculate the probability of some sequences of words from the vocabulary.

Before running the code, predict which of these sequences will have higher / lower probability.

<s> to be or not to be </s>

<s> to eat a pizza </s>

<s> to be or not to be or not to be </s>

YOUR ANSWER HERE

```

[20]: # Short sample sentence
test_seq = ["<s>", "to", "be", "or", "not", "to", "be", "</s>"]
test_prob = sequence_prob_bigram(test_seq)

assert abs(test_prob - 0.0625) < 1e-9, f"Expected 0.0625: got {test_prob}"
print(f"Test sentence probability: {test_prob}")

# Long sentence
test_seq_long = ["<s>", "to", "be", "or", "not", "to", "be", "or", "not", "to",
↪"be", "or", "not", "to", "be", "or", "not", "to", "be", "or", "not", "to",
↪"be", "or", "not", "to", "be", "or", "not", "to", "be", "or", "not", "to",
↪"be", "or", "not", "to", "be", "or", "not", "to", "be", "or", "not", "to",
↪"be", "or", "not", "to", "be", "or", "not", "to", "be", "or", "not", "to",
↪"be", "or", "not", "to", "be", "or", "not", "to", "be", "or", "not", "to",
↪"be", "</s>"]

```

```

print(f"Long sentence probability: {sequence_prob_bigram(test_seq_long)}")
print("Notice: probably can become extremely small quickly for long sentences.")

# Short sample sentence with bigram not in corpus
unseen_seq = ["<s>", "to", "eat", "a", "pizza", "</s>"]
unseen_prob = sequence_prob_bigram(unseen_seq)

assert abs(unseen_prob) < 1e-9, f"Expected 0: got {unseen_prob}"
print(f"Sentence with unseen bigram probability: {unseen_prob}")

```

Test sentence probability: 0.0625
 Long sentence probability: 1.4551915228366852e-11
 Notice: probably can become extremely small quickly for long sentences.
 Sentence with unseen bigram probability: 0.0

Log-probability of a sequence of tokens Log-probability is a similar measure of likelihood that helps us avoid dealing with numbers that are very close to 0.

Instead of multiplying probabilities:

$$P(w_1, w_2, w_3, \dots, w_n) = \prod_{k=1}^n P(w_k \mid w_{k-1})$$

we can sum logs:

$$\log P(w_1, w_2, w_3, \dots, w_n) = \sum_{k=1}^n \log P(w_k \mid w_{k-1})$$

Benefits of log-probability - avoids underflow (numbers becoming 0 in a computer) - turns products into sums (easier to compute) - maintains ordering given by probability

```

[21]: import math

def sequence_logprob_bigram(tokenized_sequence):
    total = 0.0
    for i in range(len(tokenized_sequence)-1):
        # Get the previous word and current word to form the key for bigram
        ↪probabilities
        prev_word = tokenized_sequence[i]
        curr_word = tokenized_sequence[i + 1]

        # Get the bigram probability for the current bigram (prev_word,
        ↪curr_word)
        p = bigram_probs[(prev_word, curr_word)]

        # If the bigram probability is zero, we return negative infinity for
        ↪the log-probability of the sequence

```

```

        if p == 0.0:
            return float("-inf")

        # Otherwise, we add the log of the bigram probability to the total
        ↪ log-probability of the sequence
        total += math.log(p)

    return total

print("Test Sequence Log Probability:", sequence_logprob_bigram(test_seq))
print("Long Sequence Log Probability:", sequence_logprob_bigram(test_seq_long))

```

Test Sequence Log Probability: -2.7725887222397816

Long Sequence Log Probability: -24.953298500158034

Sparsity problem (zero probabilities) If a bigram never occurred in training:

$$\hat{P}(w_k | w_{k-1}) = 0$$

Then any sentence containing it gets probability 0 (log probability = $-\infty$).

That is too harsh for real language.

```

[22]: # Tokenized sequence `<s> to eat a pizza </s>`
test_seq_2 = ["<s>", "to", "eat", "a", "pizza", "</s>"]

print(sequence_prob_bigram(test_seq_2))
print("Unseen Bigram Sequence Log Probability:",
      ↪ sequence_logprob_bigram(test_seq_2))

```

0.0

Unseen Bigram Sequence Log Probability: -inf

Laplace Smoothing (or Add-1 Smoothing) **Smoothing** is a technique that assigns non-zero probability to unseen events by redistributing probability mass from seen events.

In Laplace or Add-1 smoothing, we fix zeros by adding 1 to every possible next word:

$$P_{smooth}(w_k | w_{k-1}) = \frac{\text{count}(w_{k-1}, w_k) + 1}{\text{count}(w_{k-1}) + |V|}$$

- $|V|$ = vocabulary size
- For a given previous word w_{k-1} , there are $|V|$ possible next words
- Adding 1 to the denominator for each possible next word ensures probabilities sum to 1
- This guarantees **non-zero** probability everywhere

```

[23]: # Let's add 1 to all bigram counts to perform add-one smoothing, and then
      ↪ recompute the bigram probabilities and sentence log probabilities

```

```

# 1. Add 1 to all bigram counts
bigram_counts_smoothed = {}
for (prev_word, curr_word) in bigram_counts:
    bigram_counts_smoothed[(prev_word, curr_word)] = bigram_counts[(prev_word,
↪curr_word)] + 1

# Print smoothed bigram counts as a table
print("Smoothed Bigram Counts:")
print(f"{'':>7}", end="")
for curr_word in vocab_order:
    print(f"{curr_word:>7}", end="")
print()
for prev_word in vocab_order:
    print(f"{prev_word:>7}", end="")
    for curr_word in vocab_order:
        print(f"{bigram_counts_smoothed[(prev_word, curr_word)]:>7}", end="")
    print()

# 2. Add |V| to all word counts to account for the added counts in bigrams
word_counts_smoothed = {}
for prev in word_counts:
    word_counts_smoothed[prev] = word_counts[prev] + vocab_size # Add 1 for
↪each possible current word

# 3. Recompute bigram probabilities with smoothing
bigram_probs_smoothed = {}
for (prev_word, curr_word) in bigram_counts_smoothed:
    bigram_probs_smoothed[(prev_word, curr_word)] =
↪bigram_counts_smoothed[(prev_word, curr_word)] /
↪word_counts_smoothed[prev_word]

# Print smoothed bigram probabilities as a table
print("\nSmoothed Bigram Probabilities:")
print(f"{'':>7}", end="")
for curr_word in vocab_order:
    print(f"{curr_word:>7}", end="")
print()
for prev_word in vocab_order:
    print(f"{prev_word:>7}", end="")
    for curr_word in vocab_order:
        print(f"{bigram_probs_smoothed[(prev_word, curr_word)]:>7.2f}", end="")
    print()

```

Smoothed Bigram Counts:

	</s>	<s>	a	be	eat	king	not	or	pizza	to
</s>	1	1	1	1	1	1	1	1	1	1
<s>	1	1	1	1	1	1	1	1	1	4

a	1	1	1	1	1	2	1	1	1	1
be	2	1	2	1	1	1	1	2	1	1
eat	1	1	1	1	1	1	1	1	2	1
king	2	1	1	1	1	1	1	1	1	1
not	1	1	1	1	1	1	1	1	1	2
or	1	1	1	1	1	1	2	1	1	1
pizza	2	1	1	1	1	1	1	1	1	1
to	1	1	1	4	2	1	1	1	1	1

Smoothed Bigram Probabilities:

	</s>	<s>	a	be	eat	king	not	or	pizza	to
</s>	0.08	0.08	0.08	0.08	0.08	0.08	0.08	0.08	0.08	0.08
<s>	0.08	0.08	0.08	0.08	0.08	0.08	0.08	0.08	0.08	0.31
a	0.09	0.09	0.09	0.09	0.09	0.18	0.09	0.09	0.09	0.09
be	0.15	0.08	0.15	0.08	0.08	0.08	0.08	0.15	0.08	0.08
eat	0.09	0.09	0.09	0.09	0.09	0.09	0.09	0.09	0.18	0.09
king	0.18	0.09	0.09	0.09	0.09	0.09	0.09	0.09	0.09	0.09
not	0.09	0.09	0.09	0.09	0.09	0.09	0.09	0.09	0.09	0.18
or	0.09	0.09	0.09	0.09	0.09	0.09	0.18	0.09	0.09	0.09
pizza	0.18	0.09	0.09	0.09	0.09	0.09	0.09	0.09	0.09	0.09
to	0.07	0.07	0.07	0.29	0.14	0.07	0.07	0.07	0.07	0.07

[24]: *# Now we can compute the probability and log-probability of sequences using the*
↪ smoothed bigram probabilities

```
def sequence_prob_bigram_smoothed(tokenized_sequence):
    prob = 1.0
    for i in range(len(tokenized_sequence)-1):
        prev_word = tokenized_sequence[i]
        curr_word = tokenized_sequence[i + 1]
        prob *= bigram_probs_smoothed[(prev_word, curr_word)]

    return prob

def sequence_logprob_bigram_smoothed(tokenized_sequence):
    total = 0.0
    for i in range(len(tokenized_sequence)-1):
        prev_word = tokenized_sequence[i]
        curr_word = tokenized_sequence[i + 1]

        p = bigram_probs_smoothed[(prev_word, curr_word)]
        if p == 0.0:
            return float("-inf")
        total += math.log(p)
    return total
```

```

print("Test Sequence Smoothed Probability:", □
    ↳sequence_prob_bigram_smoothed(test_seq))
print("Long Sequence Smoothed Probability:", □
    ↳sequence_prob_bigram_smoothed(test_seq_long))
print("Unseen Bigram Sequence Smoothed Probability:", □
    ↳sequence_prob_bigram_smoothed(test_seq_2))
print()
print("*"*20)
print()
print("Test Sequence Smoothed Log Probability:", □
    ↳sequence_logprob_bigram_smoothed(test_seq))
print("Long Sequence Smoothed Log Probability:", □
    ↳sequence_logprob_bigram_smoothed(test_seq_long))
print("Unseen Bigram Sequence Smoothed Log Probability:", □
    ↳sequence_logprob_bigram_smoothed(test_seq_2))

```

Test Sequence Smoothed Probability: 1.9652982075175272e-05
 Long Sequence Smoothed Probability: 7.764615925439328e-51
 Unseen Bigram Sequence Smoothed Probability: 6.604965282651233e-05

Test Sequence Smoothed Log Probability: -10.837281471612416
 Long Sequence Smoothed Log Probability: -115.38226274959347
 Unseen Bigram Sequence Smoothed Log Probability: -9.625103783232127

Add- α Smoothing

Laplace smoothing is a special case where we add 1 to every count.

A more flexible version is to add a small positive value α :

$$P_{\alpha}(w \mid c) = \frac{\text{count}(c, w) + \alpha}{\text{count}(c) + \alpha|V|}$$

where:

- c is the context (e.g., previous word in a bigram model)
- w is the next word
- $|V|$ is vocabulary size
- $\alpha > 0$ is the smoothing hyperparameter that controls how much we smooth the probabilities.

Strength of Smoothing:

- $\alpha = 1 \rightarrow$ strong smoothing (Laplace or Add-1)
- $\alpha = 0.1 \rightarrow$ mild smoothing
- $\alpha \rightarrow 0 \rightarrow$ approaches MLE (unsmoothed counts)

NOTE: Add-1 and Add- α are simple smoothing methods. They work reasonably well for small models or text classification tasks.

For large-vocabulary language models, more advanced smoothing methods usually perform much better.

Today we saw that an **n-gram model** approximates $P(w_k \mid w_1, \dots, w_{k-1})$ using only the last $n - 1$ words (Markov assumption). MLE gives **relative frequency estimates**: $P(w_k \mid w_{k-1}) = C(w_{k-1}, w_k) / C(w_{k-1})$. Using *Log-probability avoids numerical underflow when multiplying many small probabilities Smoothing** (add-1, add-) assigns non-zero probability to unseen n-grams.

Next time, we'll see how to measure how well the model fits test data.

Limitation: N-gram models treat every word as a discrete symbol with no relationship to any other word. “King” and “queen” are just two different tokens. This is what **word embeddings** (next session) will fix.